Exercise 12 - 02./03.02.2026

Learning goals

This is the last exercise for this course. There will be no new exercises next week, but time to catch up and ask me anything to prepare for the exam. Feel free to ask me questions already and work on the exercise the other week. Depending on if you submitted already 10 complete exercises, it is optional to work on it.

The goal is getting some hands-on experience with **Webcomponents** to deepen the knowledge. Webcomponents are a set of native browser APIs, the most well known are **Custom Elements** and **Shadow DOM**. To understand the mechanics, it makes sense to create a custom element „vanilla“ - without using a frontend framework. This way we can create webcomponents to be used as widgets or microfrontends with any frontend framework.

We also want to use **@angular/elements** to wrap any component inside a custom element, allowing to create re-usable microfrontends. There are challenges of different build formats and security protection measures of external JS files. The security protection is easier to overcome and we want to see a mechanism that works inside an Angular app.

Practical Exercises

1) Create a sub-folder in your gitrepo for /webcomponents. Create a **vanilla custom element**. For this you need to create a JS file with the following base code:

```
// custom.js or hello-world.js
class HelloWorldElement extends HTMLElement {
  constructor() {
    super();
    this.attachShadow({ mode: 'open' });
  }

  // called when element added to the document
  connectedCallback() {
    this.shadowRoot.innerHTML = `<div>Hello, World!</div>`;
  }
}

// this allows any HTML file to include <hello-world> if the JS is loaded
customElements.define('hello-world', HelloWorldElement);
```

2) Create a plain ***start.html*** page. Attach the <hello-world> element and reference the src file using a <script>. The url of the JS file can be relative to your HTML file.
   Serve the folder containing your JS and HTML file. Either using ***http-server*** (you can type npm init to create a blank package.json to add scripts) or by clicking on the browser icon in your IDE (HTML editor).
   The custom element should be visible in the browser without errors.
   Inspect the HTML to see the shadow DOM being created.

3) **Add CSS** to your HTML file to style all **div** elements with ***background-color: yellow***. Add Hello directly in the start.html file to see what happens to that element compared to the injected webcomponent. Does the CSS affect our Webcomponent?
   Info: Shadow DOM is a great technique to be used together with Custom Elements ensuring a „black box“ Webcomponent with stable design. The disadvantage is you cannot reuse CSS or fonts present in the surrounding page.

4) Replace the code in our webcomponent by commenting out the ***attachShadow*** command and changing ***this.shadowRoot.innerHTML*** to ***this.innerHTML***. Does the external CSS change the style now?

5) Now go to your Angular application and add the vanilla custom element JS file from task 1) to the public assets folder (/public) and load it via the index.html from Angular.
   The src path should be ***public/custom.js***.
   Add the <hello-world></hello-world> custom element in the index.html and serve Angular.
   Inspect the browser for errors. What is the error message?

6) Angular has a lot of security mechanisms to protect for script injection. You can **get rid of the security / content-type „problem“** by attaching the <script> not via HTML (template), but using the TS part of your AppComponent. Add this and you should be able to use the <hello-world> element in your index.html „alive“:

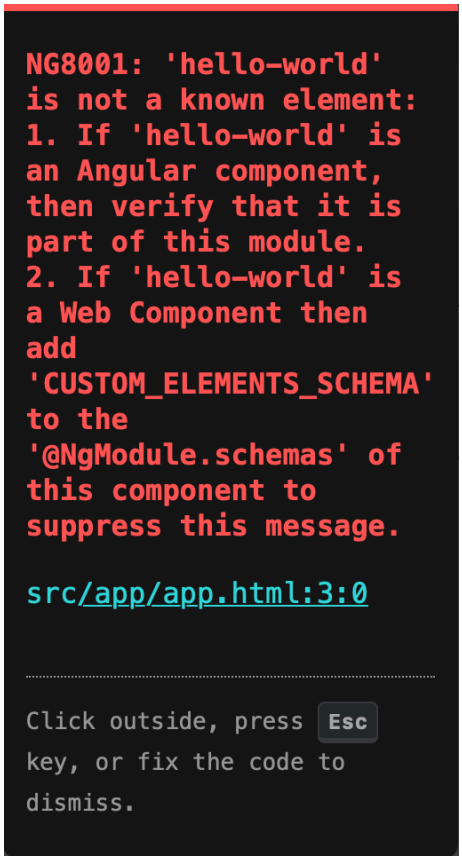
```
loadScript(url: string) {
```

```
  const body = <HTMLDivElement> document.body;
  const script = document.createElement('script');
  script.innerHTML = '';
  script.src = url;
  body.appendChild(script);
}

ngOnInit() {
  this.loadScript('custom.js');
}
```

Info: This would also allow to load scripts at run-time from a trusted remote url assuming CORS is properly handled by the webserver.

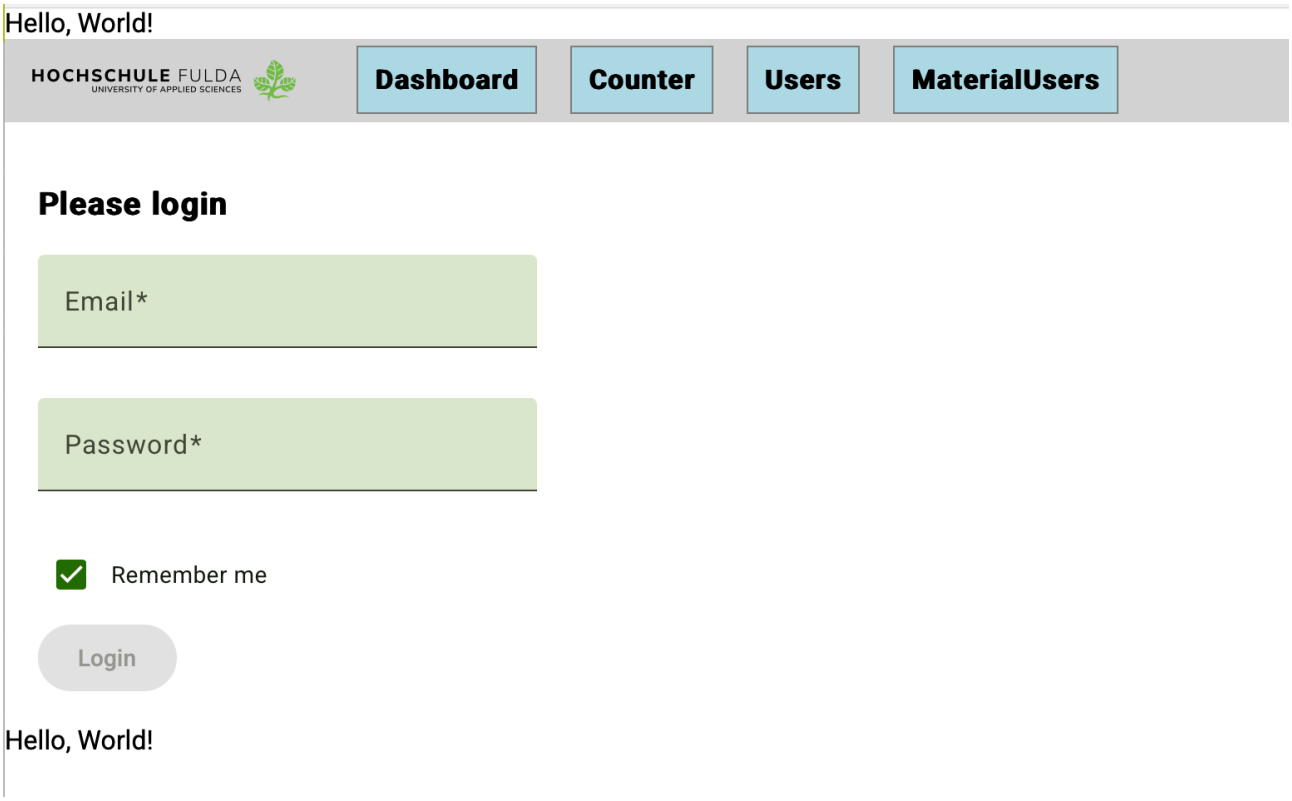
7) If you want to add the <hello-world> inside Angular, in a template, e.g. in app.component.html, **you need to tell Angular allowing unknown tags in templates**. Otherwise you see this error:



You can fix this by adding this to your @Component definition (for standalone) or by adding this to your @NgModule (non-standalone):

```
schemas: [CUSTOM_ELEMENTS_SCHEMA], // import from @angular/core
```

Add the custom element <hello-world> to a component template. You should now see two instances: one from index.html and one from the template.



8) Create a „BannerComponent“ in your Angular app using the CLI.
   The component shall take an @Input argument for a text (@Input() text: string = '';) and renders a yellow button with a width of 300px and a height of 50px, while the text is centered in the middle.
   Add the banner using <app-banner></app-banner> to the template of your AppComponent to test it. Provide the input parameter text (e.g. „Hello Banner“). Now you created a „normal“ Angular component.

9) Install Angular Elements to that Angular Application. Be sure you are using the same major version. Use ***ng add @angular/elements***

If you have trouble with peerDependencies conflicts, try to specify a distinct version like 21.1.2 and delete both node_modules and package-lock.json before re-running „npm install“.
I also recently did an Angular update in 3 steps (committing changes in between): ng

update @angular/cli => ng update @angular/core => ng update @angular/material (***ng update*** gives you that hints).
My package.json now looks like this:

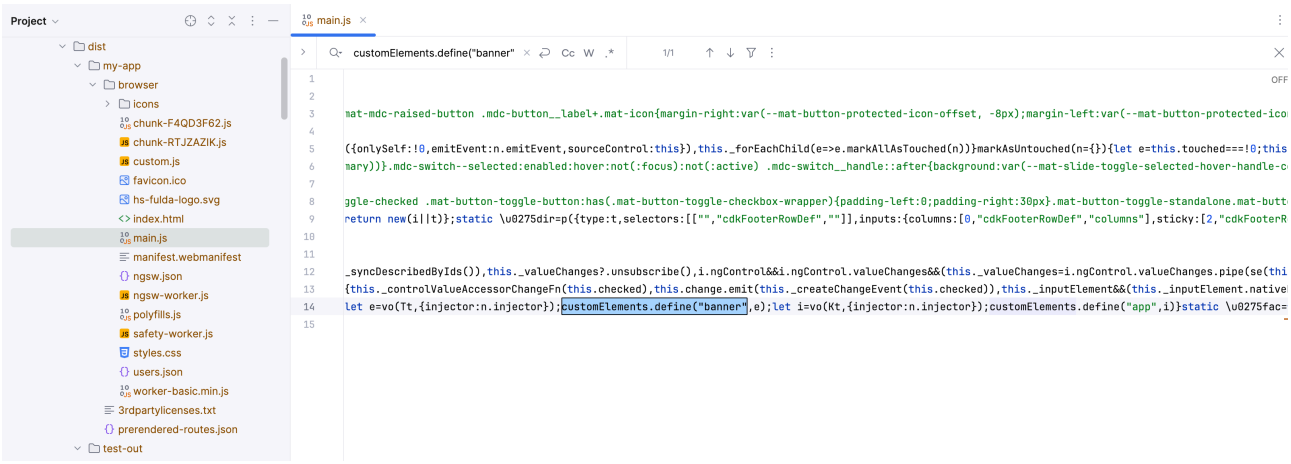
```
"dependencies": {
  "@angular/cdk": "^21.1.2",
  "@angular/common": "^21.1.2",
  "@angular/compiler": "^21.1.2",
  "@angular/core": "^21.1.2",
  "@angular/forms": "^21.1.2",
  "@angular/material": "^21.1.2",
  "@angular/platform-browser": "^21.1.2",
  "@angular/router": "^21.1.2",
  "@angular/service-worker": "^21.1.2",
  …
},
"devDependencies": {
  "@angular/build": "^21.1.2",
  "@angular/cli": "^21.1.2",
  "@angular/compiler-cli": "^21.1.2",
  "@angular/elements": "^21.1.2",
  …
}
```

10) **Create a custom element out of your BannerComponent** using Angular elements. (check the slides how to do so - either modifying app.module.ts or main.ts).
    Define your BannerComponent to be projected into <banner></banner>.
    Which command from @angular/elements do you need? Try creating a second custom element for the whole app, projecting AppComponent into <app>.
    You will not notice a difference yet, app should work as before running „ng serve“.

11) To see what the creation of the custom element is doing, you need to create a production build of Angular.
    In the distribution folder, you will then find a script that provides the new custom element for the banner tag.
    Run ***ng build* --output-hashing=none** and see JS files without a changing hash being created in the dist folder (/dist-my-app/browser in my project).

The ***main.js*** file contains your Custom Element. In theory you could take this and **add it to any external HTML file** like our start.html. In combination with the downloaded JS file you should be able to use **<banner>** and **<app>** everywhere. Check the main.js file and inspect the code to see that the vanilla API call „customElements.define“ is included.

My dist output looks like this:



In reality, there is much more effort needed to fine-tune compiler options, providing the script in the right format (esbuild/vite produce modules by default which are not plain JS).
You could try copy pasting / adding the main.js as a source file and attach the <banner> element to the start.html (outside your Angular project), but you will likely see an error.



Info: There is much more to experiment with: e.g. configuring the custom element with input parameters and receiving output events.

But getting the initial setup running is already a big barrier and a ticket for this can take a week or more in a company project.
With newer Angular versions, switching from webpack to esbuild or vite did complicate creating a „simple“ importable JS file. This is beyond the scope of this course but it helps being able to understand the complexity of big frameworks and importance to understand the root cause of errors in the browser console.